

IMPERIAL

Project KAI 9

Design History File
June 2026

Ahmad Wahab	02422484
Hisham Khatib	02391620
Jennifer Emezie	02246573
Luke Scully	02459465
Niklas Rockenfelt	06074304
Wei Xu	02413772



Contents

1	Introduction	4
1.1	Project Background	4
2	System Requirements and Specification	4
2.1	Project Objectives	4
2.2	Minimum Functional Requirements	4
2.3	Stretch Goals	5
3	Project Planning and Team Organisation	6
3.1	Initial Project Planning	6
3.2	Project Proposal Presentation	6
3.3	Mid-Project Review	6
4	Power Subsystem	7
4.1	Power Distribution	7
4.1.1	Energy source	7
4.1.2	Main protection and isolation	7
4.1.3	Branch distribution	7
4.1.4	Compute-rail redesign following buck-converter failure	8
4.1.5	Conductor sizing	8
4.1.6	Protection layers	9
4.2	Charging System	9
4.2.1	Charger	9
4.2.2	Docking station geometry	9
4.2.3	Phase 1: direct-wire charging	9
4.2.4	Phase 2: relay isolation	10
4.3	Vacuum MOSFET Switch	10
4.3.1	Purpose and topology	10
4.3.2	MOSFET selection	10
4.3.3	Gate drive network	11
4.3.4	Flyback protection	11
4.3.5	Grounding	11
4.3.6	Thermal and control behaviour	11
4.4	Bill of Materials	12
5	Architecture Redesign and Systems Integration	12
5.1	Section Overview	12
5.2	The Original Dual-Pi Architecture & The Hardware Pivot	12
5.2.1	The Initial Vision (Dual-Pi Setup)	12
5.2.2	The Hardware Failure: The Death of the Pi 5	13
5.2.3	The Pivot: Laptop (RTX 5070 Ti) Migration	13
5.3	Identified Software Integration Failures	14
5.4	The New Architecture & Solutions Implemented	14
5.5	Systems Synergy: How It All Works Together	15
6	WhatsApp, Wellbeing Assistant, and Calendar Integration	16
6.1	Purpose and Requirements	16
6.2	System Architecture	17
6.3	Command Processing, Weather, Calendar and Reminder Functions	17
6.3.1	Webhook and Command Handling	17
6.3.2	Weather Advice	17
6.3.3	Apple Calendar Event Creation	18
6.3.4	Reminder System	18
6.4	Problems Encountered and Design Changes	19
6.4.1	Raspberry Pi 5 failure	19
6.4.2	Docker and ngrok networking	19
6.4.3	Misleading event error message	19

6.5	Testing Procedure	19
6.6	Handover and Reproduction	20
6.7	Limitations and Future Work	20
7	Health & Activity Monitoring	20
7.1	Camera Vision	20
7.1.1	The Sony IMX500 AI Camera	20
7.1.2	Design Rationale	20
7.1.3	Software Architecture on the Pi 4	21
7.1.4	Integration with the Edge Node	21
7.2	Heart Rate Monitor	22
7.2.1	Decathlon HRM Chest Strap	22
7.2.2	BLE Protocol	22
7.2.3	Software Architecture on the Pi 4	22
7.2.4	Integration with the Edge Node	23
7.2.5	Future Extensions	23
8	Movement Subsystem & Navigation Integration	23
8.1	Design Evolution, URDF, & Simulation Phase	23
8.2	Hardware Sourcing & Specifications	23
8.3	Initial Software Architecture	24
8.4	Hardware Failure & Contingency Implementation	24
8.4.1	Odometry & Encoder Kinematics	24
8.4.2	Main Controller & Bug Algorithm Bypass	25
9	Design Decisions and Rationale	25
9.1	Chassis and Enclosure Design Iterations	26
9.2	Testing and Validation	27
9.3	Final Prototype Implementation	28
10	Materials and Bill of Materials	28
10.1	Overview	29
10.2	Major Components	29
10.3	Fabrication Materials	29
10.4	Power Distribution Components	29
10.5	Total System Cost	30
10.6	Bill of Materials Traceability	30
11	Future Work	30
11.1	Conclusion	31

GitHub Repository: <https://github.com/Ls3123/Kai9Project>

1 Introduction

1.1 Project Background

Population ageing and increasing levels of social isolation present significant challenges for independent living. Many elderly individuals require assistance with daily household tasks while simultaneously benefiting from regular social interaction and wellbeing monitoring. Existing robotic solutions typically focus on either domestic automation or social companionship, with few systems combining both functionalities within a single platform.

The KAI 9 project was developed to address this challenge through the creation of an autonomous sanitation and social companion robot. The system combines autonomous navigation, vacuum cleaning capabilities, voice interaction, wellbeing prompting, and remote communication tools within a single integrated platform. The objective was to develop a robotic companion capable of supporting users in their daily lives whilst promoting independence, engagement, and safety.

The project was undertaken over an eight-week development period and followed a multidisciplinary engineering approach involving mechanical design, embedded electronics, autonomous navigation, artificial intelligence, and cloud-based communication services. The final prototype incorporates a tracked drive system, LiDAR-based obstacle detection, an onboard vacuum cleaning subsystem, audio interaction capabilities, WhatsApp integration, and a self-charging docking station.

2 System Requirements and Specification

2.1 Project Objectives

At the outset of the project, a set of minimum functional requirements was established to define the Minimum Viable Product (MVP). These requirements were selected to ensure that the final system delivered both practical household functionality and meaningful user interaction.

The primary objectives of the project were:

- Autonomous movement within an indoor environment.
- Obstacle detection and avoidance using LiDAR.
- Integrated vacuum cleaning functionality.
- Audio interaction through a microphone and speaker system.
- Remote communication through WhatsApp.
- Self-charging capability via a docking station.
- Wellbeing and activity prompting features.
- Secure handling of user data and privacy protection.

2.2 Minimum Functional Requirements

The project requirements were divided between the hardware and software development teams. This division allowed both workstreams to progress in parallel whilst maintaining regular integration milestones.

Table 1: Minimum Functional Requirements

Requirement	Hardware Team	Software Team
Speaker and Microphone	Integration of microphone array, speakers and audio hardware.	Audio playback, speech processing and voice interaction services.
Autonomous Movement	Motor installation, track assembly and drivetrain integration.	Navigation logic, motion control and state management.
LiDAR Navigation	LiDAR mounting, power delivery and interfacing.	Obstacle detection, mapping and path-planning algorithms.
Tracks and Motors	Selection, mounting and testing of motors, tracks and motor drivers.	Motor control software and movement commands.
Vacuum Cleaning	Vacuum motor integration, ducting and mechanical housing.	Vacuum activation, scheduling and cleaning control logic.
WhatsApp Integration	N/A	WhatsApp API integration, messaging and notification services.
Self-Charging Station	Charging dock design, power electronics and charging contacts.	Battery monitoring and docking behaviour.
Activity Prompts	Audio output hardware support.	Wellbeing monitoring, reminders and activity prompting.
Privacy and Security	Physical protection of hardware and secure storage.	Authentication, credential management and access control.

2.3 Stretch Goals

In addition to the core requirements, a number of advanced features were identified during project planning. These features were classified as stretch goals and would only be implemented if sufficient development time remained after completion of the MVP.

These optional enhancements included:

- Advanced conversational artificial intelligence.
- Live life updates and contextual awareness.
- Voice calling functionality.
- Long-term event logging.
- Learning user routines and behavioural patterns.
- Alarm clock functionality.
- Medication reminders.
- Google Calendar integration.
- Personalised voice synthesis.
- Companion mobile application.

Several of these features were partially investigated during development and remain potential candidates for future expansion of the platform.

3 Project Planning and Team Organisation

3.1 Initial Project Planning

The first phase of the project focused on requirements analysis, architecture planning, component sourcing and task allocation. During the initial project meetings, the team reviewed the project brief and identified the major technical challenges associated with integrating autonomous navigation, cleaning functionality and conversational interaction within a single robotic platform.

To improve development efficiency, the team was divided into two specialist groups: a Hardware Team and a Software Team. This division allowed mechanical, electrical and software development activities to progress in parallel while maintaining regular integration milestones.

The hardware team focused primarily on chassis design, drivetrain integration, power systems, sensor mounting and vacuum implementation. Simultaneously, the software team concentrated on navigation algorithms, artificial intelligence integration, communication services and user interaction systems.

During the first two weeks of development, significant effort was dedicated to researching suitable components, sourcing hardware, evaluating suppliers and establishing development environments. Team deadlines and milestone targets were agreed upon to ensure progress remained aligned with the project schedule.

3.2 Project Proposal Presentation

Before major development commenced, the team presented a formal project proposal to the client. The purpose of this presentation was to validate the project direction and ensure that the proposed system architecture aligned with stakeholder expectations.

The proposal outlined:

- The intended system architecture.
- Hardware and software responsibilities.
- Project timeline and deliverables.
- Budget considerations.
- Technical risks and mitigation strategies.

Feedback received during the proposal stage confirmed that the selected project scope was appropriate and that the planned functionality met the expectations of the client.

3.3 Mid-Project Review

A formal progress review was conducted during Week 4 of the project. The purpose of this review was to evaluate whether development remained on track and identify any emerging risks that could impact final delivery.

The review assessed:

- Progress against planned milestones.
- Hardware procurement status.
- Software development progress.
- Integration readiness.
- Budget utilisation.
- Technical risks and contingency plans.

This review provided an opportunity to identify delays, adjust priorities and ensure that development effort remained focused on achieving a robust and demonstrable Minimum Viable Product. The findings of the review informed several subsequent design decisions and contributed significantly to the successful completion of the project.

The final prototype successfully achieved all primary Minimum Viable Product requirements established during the project planning phase. Core functionality including autonomous movement, obstacle avoidance, vacuum cleaning, voice interaction, remote communication, wellbeing prompting and self-charging capability were demonstrated successfully.

Several stretch-goal features were either partially implemented or intentionally deferred in order to prioritise system stability and successful delivery of the core platform. This decision was made following the Week 4 project review and subsequent hardware setbacks, including the failure of the Raspberry Pi 5 compute platform. Prioritising robustness over feature expansion ensured that all critical project objectives were met within the available timeframe.

4 Power Subsystem

4.1 Power Distribution

4.1.1 Energy source

The energy source is an Azeco LP12V20 LCD battery: a 12.8 V nominal, 20 A h lithium iron phosphate (LiFePO_4) pack with an integrated battery management system (BMS) and an LCD state-of-charge display, terminated in M5 studs. LiFePO_4 chemistry is chosen for its flat discharge curve, long cycle life and superior thermal stability compared with lithium-polymer. Although the nominal terminal voltage is 12.8 V, this is fully within the tolerance of accessories rated for nominal “12 V” automotive/DC operation, so no intermediate regulation is required for the 12 V rail.

4.1.2 Main protection and isolation

Immediately downstream of the battery positive terminal, and within 150 mm of it, sits a 30 A ATO blade fuse in an RS PRO inline holder. This main fuse exists to protect the short, unbranched wire segment between the battery and the fuse block, a segment that no branch fuse can protect. Without it, a fault in that segment (for example, chafed insulation against the chassis) would be limited only by the cell chemistry and could deliver hundreds of amps into a dead short.

After the main fuse, a master kill switch (Switch Electronics 550453, a 30 A 12 V DC illuminated toggle) provides single-action disconnection of all loads. It is placed on the battery-positive output *before* load distribution, so that flipping it de-energises the entire robot for safe handling, transport and servicing.

4.1.3 Branch distribution

The switched positive feed enters a Blue Sea 5025 six-way fuse block with a common negative busbar. Five branches are active; one position is left spare for future expansion. The branch allocation is summarised in Table 2.

Table 2: Fuse block branch allocation.

Branch	Fuse	Load	Output
F1	3 A	DFR1025 buck converter → Pi 4	5.1 V rail
F2	5 A	DFR1025 buck converter → Pi 5	5.1 V rail
F3	20 A	Cytron MDD10A dual motor driver	12 V
F4	10 A	12 V vacuum (via IRLZ44N switch)	12 V
F5	5 A	ZK-502C audio amplifier	12 V
F6	–	Spare	–

F1 and F2: independent compute rails. Each single-board computer is fed by its own DFR1025 buck converter on its own fuse branch: F1 (3 A) supplies the Raspberry Pi 4 and F2 (5 A) supplies the Raspberry Pi 5. Each converter is configured for a 5.1 V output (set via the module’s onboard adjustment; verified at 5.15 V no-load), the slight elevation above 5.0 V compensating for resistive drop along the supply wiring so that each Pi sees a healthy 5 V under load. The branch fuses sit on the 12 V input side: the reflected input current

is approximately 1.5 A for the Pi 4 and 2.5 A for the Pi 5, so the 3 A and 5 A ratings provide ample margin for capacitor inrush at power-up. This split topology is a direct response to a field failure of the original shared-rail design, discussed in Section 4.1.4.

F3: drivetrain. A 20 A fuse feeds the Cytron MDD10A dual motor driver. The driver is rated 10 A continuous per channel with substantial peak headroom; the 20 A fuse passes normal running and stall-recovery currents without nuisance blowing while still protecting the wiring.

F4: vacuum. A 10 A fuse feeds the vacuum motor through the MOSFET switch described in Section 4.3. The vacuum draws ~ 4 A running; the 10 A rating tolerates start-up inrush.

F5: audio. A 5 A fuse feeds the ZK-502C audio amplifier, which drives the Visaton FR10 speaker for the voice companion and media functions.

4.1.4 Compute-rail redesign following buck-converter failure

The compute rail was originally implemented as a *single* DFR1025 buck converter supplying both the Pi 4 and the Pi 5 from one shared 5.1 V rail, so that one regulator and one grounding scheme served the whole compute cluster. During integration this design failed destructively: the Raspberry Pi 5 was lost to an over-voltage event traced to the compute-rail converter. The symptom—instantaneous, unrecoverable failure of the board—is consistent with the converter’s series pass element failing short-circuit and presenting the unregulated 12 V input directly on the regulated 5 V output. Because the hardware had to be returned before a post-mortem could be performed, the underlying cause of the converter fault itself could not be confirmed; the failure *mode* is therefore inferred from the symptom rather than measured directly.

Two design weaknesses were identified:

1. **Shared rail as a single point of failure.** Both computers hung off one converter, so a single pass-through fault placed the full 12 V input on *both* boards simultaneously. The blast radius of one component was the entire compute cluster.
2. **Fusing does not protect against this mode.** The branch fuse sits on the 12 V input and protects the wiring against over-current. A pass-through short delivers an over-voltage at the modest current the downstream board draws before it fails (a few amperes)—well below the fuse rating—so the fuse never clears. Over-current protection and over-voltage protection are distinct functions, and the former does not provide the latter.

The implemented redesign gives each Pi its own DFR1025 converter on its own fuse branch (F1 $\rightarrow$ Pi 4, F2 $\rightarrow$ Pi 5). A converter failure is now contained to a single board rather than taking out the whole cluster. As a secondary benefit, each converter now operates at a small fraction of its rating instead of near the continuous ceiling of a single shared unit, which removes the earlier thermal concern and the need for heatsinking on the compute rail.

It should be noted that this topology limits the *blast radius* of a converter fault but does not, by itself, prevent over-voltage from reaching a board if that board’s own converter fails. Full protection against pass-through over-voltage requires a per-rail over-voltage protection (OVP) crowbar—a TVS- or zener-triggered SCR, or a dedicated OVP controller, clamping each 5 V rail at ~ 5.5 V. This is recommended as future hardening for any unattended deployment and is recorded in the future-work section.

4.1.5 Conductor sizing

Tri-rated (BS 6231) wire is used throughout for its thin wall and high temperature rating. Two gauges are deployed:

- **12 AWG** on the main feed, the F3 motor branch and the negative return.
- **16 AWG** on all remaining branches (F1, F2, F4, F5) and the charging path.

The choice of 12 AWG on the 30 A-fused main feed is conservative: free-air and ventilated chassis runs of this short length comfortably carry the realistic peak draw of ~ 20 A to 22 A, and the 30 A fuse will clear long before the conductor is thermally stressed. On the F3 branch, 16 AWG is used *at the driver terminals* as a documented and deliberate trade-off, because 12 AWG will not physically fit the MDD10A terminal blocks; the short length and 20 A fuse keep this within safe limits.

4.1.6 Protection layers

The distribution network provides the following independent layers of fault protection:

1. **Internal BMS:** cell-level over-current, over-voltage, under-voltage and short-circuit protection inside the battery.
2. **30 A main fuse:** protects the unbranched feed segment.
3. **Branch fuses:** isolate individual load faults (F1–F5).
4. **Independent compute converters:** a buck-converter fault is contained to a single Pi rather than the whole compute cluster (Section 4.1.4).
5. **Phase 2 relay isolation:** disconnects the charge contacts when undocked (Section 4.2).

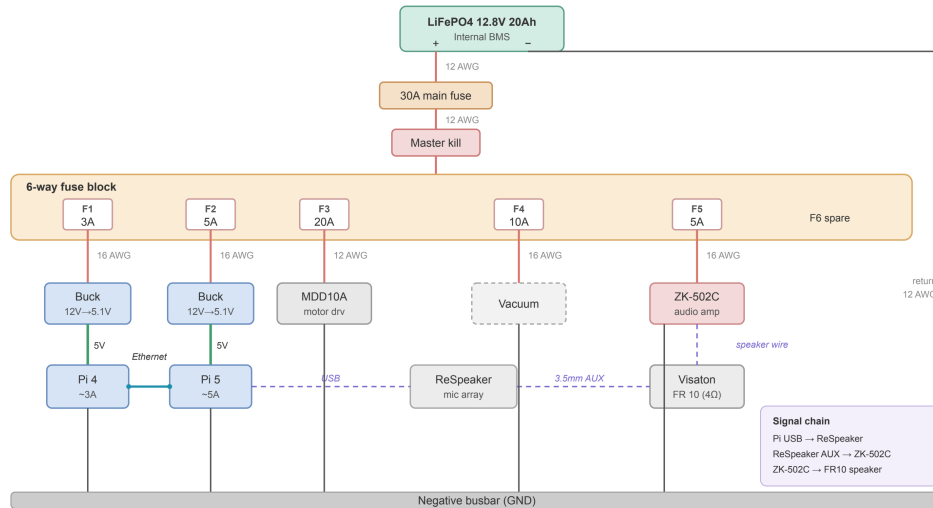


Figure 1: Full power distribution: battery → main fuse → master kill → six-way fuse block → branches (two independent buck/compute rails, motor driver, vacuum, audio), with shared negative busbar.

4.2 Charging System

4.2.1 Charger

Recharging is performed by an Eco-Worthy LiFePO₄ charger with a user-selectable 5 A/10 A output. The charger produces the correct 14.6 V absorption voltage for a four-cell LiFePO₄ pack and manages the full charge profile, so the robot's onboard electronics perform no charge regulation. A full charge from a typical depth of discharge completes in approximately three hours at the 5 A setting.

4.2.2 Docking station geometry

The dock consists of a circular disc base ($\sim \varnothing 200 \text{ mm} \times 30 \text{ mm}$) joined to a cuboid charger housing ($\sim 210 \times 100 \times 70 \text{ mm}$ external). Charge transfer is via spring-loaded pogo pins on the dock mating with copper contact plates ($\sim 60 \text{ mm}$) on the underside of the chassis. The pins are spaced $\sim 90 \text{ mm}$ centre-to-centre to match the chassis plate layout, giving a tolerant, self-aligning engagement as the robot drives onto the dock. The pogo contacts are rated for the charge current (12 A contact rating) with comfortable margin.

4.2.3 Phase 1: direct-wire charging

The charging system is commissioned in two phases to de-risk the live-contact design. In Phase 1 (complete), the dock is hard-wired through a 15 A fuse on 16 AWG directly to the battery, with no logic-controlled isolation. This minimal form is used to validate the difficult mechanical and electrical fundamentals first: contact polarity, pin/plate alignment, plate-to-fuse junction integrity, charge-profile behaviour and long-term contact pitting across repeated docking cycles. Spade connectors are used at the plate-to-fuse junction so that the Phase 2 relay can later be inserted cleanly without rework.

4.2.4 Phase 2: relay isolation

The limitation of Phase 1 is that the chassis copper plates are permanently tied to the battery, so they are live whenever the robot is undocked, creating an exposed-contact hazard. In Phase 2, a logic-controlled SPST 40 A automotive relay is inserted between the charge-plate positive and the battery positive. The relay is energised only when the robot detects that it is correctly docked, so the external contacts are dead at all other times. This isolation is referenced in the distribution section and is the recommended final configuration for any demonstration involving an accessible dock.

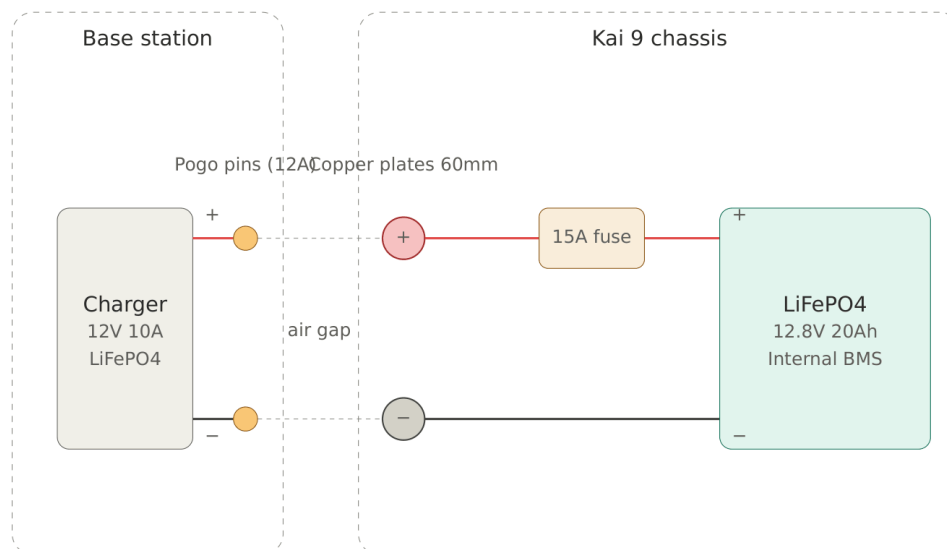


Figure 2: Charging system: base station charger and pogo pins mating with chassis copper plates, through the in-line charge fuse to the battery.

4.3 Vacuum MOSFET Switch

4.3.1 Purpose and topology

The vacuum motor is a commercial off-the-shelf 12 V handheld unit switched on and off in software. Because the Raspberry Pi cannot source motor current from a GPIO pin, the load is gated by a *low-side N-channel MOSFET*: the motor's positive terminal connects permanently to the fused 12 V rail (F4), and the MOSFET sits between the motor's negative terminal and the ground busbar. Driving the gate high connects the motor to ground and the motor runs; driving it low disconnects it. A low-side switch is chosen over a high-side arrangement because it allows the gate to be driven directly from the 3.3 V GPIO logic with no level-shifting or gate-driver IC.

4.3.2 MOSFET selection

The switching device is an IRLZ44N. The critical requirement is that the device be a *logic-level* MOSFET, one that turns fully on at a gate-source voltage available from the GPIO. The IRLZ44N reaches a very low on-resistance ($R_{DS(on)} \approx 22 \text{ m}\Omega$) at $V_{GS} = 4 \text{ V}$ and is already substantially enhanced at 3.3 V, making it suitable for direct GPIO drive.

Several alternatives were explicitly evaluated and rejected:

- **IRF530 / IRF9530**: standard-threshold MOSFETs that do not fully enhance at 3.3 V; they would operate in a partially-on, high-dissipation state.
- **TIP102 / TIP41C**: bipolar transistors requiring continuous base current or exhibiting excessive saturation power loss.
- **BZX85 / Zener series**: the wrong device class entirely for a flyback or switching role.

4.3.3 Gate drive network

The gate is driven from GPIO18 (physical pin 12) through a $220\ \Omega$ series resistor, which limits the transient current that charges the gate capacitance and tames switching edges. A $10\ \text{k}\Omega$ resistor between gate and source holds the gate low whenever the GPIO is undriven (for example during boot, before pin configuration), guaranteeing the motor stays off and cannot start unexpectedly.

4.3.4 Flyback protection

The vacuum motor is an inductive load, so interrupting its current generates a large reverse voltage spike that would otherwise stress the MOSFET drain. An SB560 Schottky diode (5 A, 60 V) is placed across the motor as a flyback path: cathode to 12 V, anode to the MOSFET drain (the motor's negative terminal). When the MOSFET turns off, the collapsing field drives current harmlessly around this loop. A Schottky type is chosen for its fast recovery and low forward drop.

4.3.5 Grounding

All grounds (GPIO drive return, gate pulldown, MOSFET source and the motor's flyback loop) terminate at the shared negative busbar. In addition, a dedicated ground jumper is run from the Pi's ground (physical pin 14) directly to the busbar rather than relying solely on the USB-C path through the buck converter. This gives the gate-drive signal a clean, low-impedance return, which matters because the motor is switched with a 20 kHz PWM waveform.

4.3.6 Thermal and control behaviour

At the running current of 4 A, conduction loss in the MOSFET is $P = I^2 R_{DS(on)} \approx 4^2 \times 0.022\ \Omega \approx 0.35\ \text{W}$, which the TO-220 package dissipates comfortably in free air; a heatsink is therefore optional. The motor is not switched as a hard on/off step. Instead a 20 kHz PWM soft-start ramp brings the duty cycle up over a short interval, limiting inrush current and mechanical stress at start-up. 20 kHz is chosen to keep switching above the audible band.

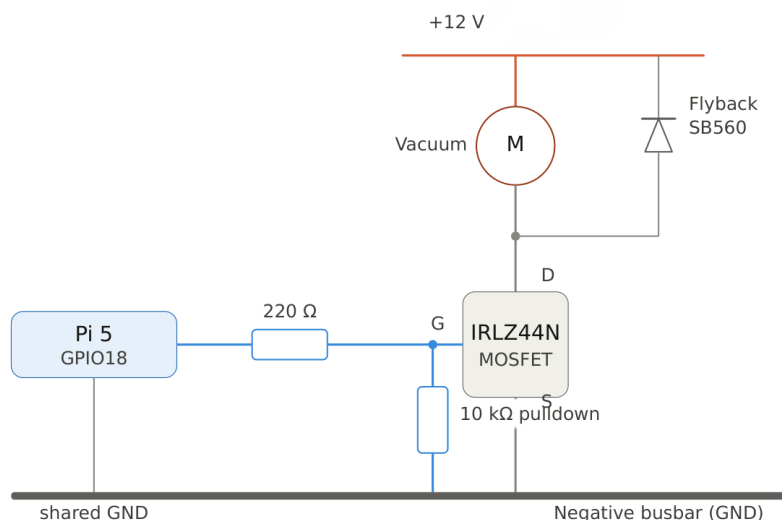


Figure 3: Vacuum low-side switch: GPIO18 $\rightarrow$ $220\ \Omega$ gate resistor $\rightarrow$ IRLZ44N gate, with $10\ \text{k}\Omega$ pulldown; motor on the 12 V (F4) rail with SB560 flyback diode; source and all returns to the negative busbar.

4.4 Bill of Materials

Table 3: Power subsystem bill of materials (key items).

Component	Part / model	Fuse	Wire
Battery	Azeco LP12V20 LCD, 12.8 V 20 A h LiFePO ₄ (BMS)	-	12 AWG
Charger	Eco-Worthy 10 A LiFePO ₄ , 14.6 V out	15 A (chg)	16 AWG
Dock contacts	Pogo pins + $\varnothing$ 60 mm Cu plates	-	-
Main fuse	30 A ATO blade + RS PRO inline holder	30 A	12 AWG
Master kill	Switch Electronics 550453 toggle	-	12 AWG
Fuse block	Blue Sea 5025 six-way + busbar	-	-
F1 buck (Pi 4)	DFR1025 buck 5.1 V $\rightarrow$ Pi 4	3 A	16 AWG
F2 buck (Pi 5)	DFR1025 buck 5.1 V $\rightarrow$ Pi 5	5 A	16 AWG
F3 motor driver	Cytron MDD10A	20 A	12 AWG
F4 vacuum	12 V handheld + IRLZ44N switch	10 A	16 AWG
F5 audio amp	ZK-502C	5 A	16 AWG
Speaker	Visaton FR10, 4 Ω	-	spkr
MOSFET	IRLZ44N (logic-level)	-	-
Flyback diode	SB560 Schottky, 5 A 60 V	-	-
Phase 2 relay	SPST 40 A automotive	-	12 AWG

5 Architecture Redesign and Systems Integration

5.1 Section Overview

This section details the architectural evolution of the "Kai" (or Scooby) robotic system, an autonomous edge-computing platform designed as an advanced eldercare companion. Initially conceived as a completely untethered "Dual-Pi" system (Raspberry Pi 5 and Raspberry Pi 4), the project faced a critical hardware failure when the primary compute node (the Pi 5) was destroyed by an electrical fault. This necessitated a rapid architectural pivot, offloading the cognitive and navigational processing to a local Windows Laptop equipped with an RTX 5070 Ti.

Following this hardware migration, several software integration challenges emerged between the new laptop Brain and the Pi 4 Edge node—specifically regarding ZMQ communications, LLM logic coupling, and real-time audio processing in loud environments. This overview outlines the robot's mission profile, the original vision, the hardware pivot, the software integration failures, and the subsequent redesign that led to a successful, robust demonstration. A foundational feature consistent across both the original and final architectures is a highly efficient, hierarchical "Global Ledger" memory system that grants the robot boundless long-term recall without overwhelming system memory.

5.2 The Original Dual-Pi Architecture & The Hardware Pivot

5.2.1 The Initial Vision (Dual-Pi Setup)

The original blueprint for Kai was designed to be a 100% onboard, fully untethered, and autonomous mobile robot. The chassis components and logic boards were powered centrally by a high-capacity LiFePo₄ battery, stepped down via buck converters to provide the specific required voltages.

To achieve full autonomy without relying on cloud processing, the compute architecture was strictly divided across two Raspberry Pi single-board computers. The software ecosystem was highly modular, utilizing Docker containers across both boards to ensure clean environments and isolated dependency management. To facilitate debugging across the multi-pi architecture, containers were started manually, and physical hardware passthrough was strictly handled via explicit device mapping (e.g., `-device=/dev/ttyUSB0`).

While the two boards communicated via a direct Ethernet connection to provide a dedicated, ultra-low-latency ZeroMQ (ZMQ) pipeline, the system was bootstrapped and managed headlessly via SSH over a local Wi-Fi network. This distributed edge computing model isolated critical, real-time physical safety protocols from the heavy, unpredictable processing loads of artificial intelligence.

- The "Upper Brain" (Raspberry Pi 5): Functioning as the primary cognitive and navigational host, the Pi 5 ran on Ubuntu, a choice specifically driven by the ease of deploying Robot Operating System 2 (ROS 2) and its associated AI tooling within that environment.
 - Cognitive Routing & Task Management (Tier 1): Tasked with running a 1-Billion parameter Llama 3.0 model. Due to the strict memory and CPU limits of the board, this original cognitive setup could not support a heavy multi-agent orchestration framework. Instead, it relied on a highly efficient, custom OPCODE-based task queue system. Every incoming request or internal system action was assigned a specific OPCODE value and slotted into a dynamic task list. The system processed these tasks one by one based on seniority, continuously executing and draining the queue until no tasks remained. This lightweight, linear architecture prevented CPU bottlenecks while ensuring all user commands and logic steps were handled systematically.
 - Hierarchical Memory & Global Ledger: To provide long-term contextual awareness without exhausting the Pi 5's limited RAM, a two-stage memory architecture was designed. Tasks appended brief summaries to a rolling short-term log. Every 30 minutes, an asynchronous daemon prompted the LLM to summarize this log, storing it as a chronological entry in an external JSON "Global Ledger" before flushing the active RAM.
 - Navigation Engine & LiDAR (Tier 2): Handled the dense mathematical pathfinding. Crucially, the Pi 5 physically managed the RPLidar scanner via USB. The LiDAR processing required a strict minimum of 2 GB of RAM, making the 16 GB Pi 5 the mandatory host for this sensor in the original blueprint. It ingested these 360-degree LiDAR arrays to calculate differential drive kinematics and execute Bug Algorithm bypass maneuvers.
- The "Lower Brain" (Raspberry Pi 4): Designated as Tier 3 (The Edge Hardware Portal), this node ran Raspberry Pi OS to ensure maximum compatibility and stability with low-level GPIO hardware interfaces. It acted as the physical proxy, reflex center, and primary audio ingestion node.
 - Audio & Sensory Input: Handled real-time continuous audio processing (RealtimeSTT and Text-to-Speech). It utilized a connected ReSpeaker microphone array, relying on its built-in hardware filtration to isolate human speech from the ambient noise of the vacuum motors, and transmitted the transcribed text up to the Pi 5 via ZMQ. Additionally, it managed the warm-booted IMX500 AI Vision camera mounted at the "eyes" of the robot, performing on-silicon object and stance detection without taxing the main CPUs.
 - Actuation & Safety: Directly controlled the Cytron motor drivers, wheel odometry encoders, and the 12V vacuum relay via GPIO. Crucially, it featured a hardcoded hardware reflex loop: if proximity data detected obstacles within 15cm, the Pi 4 would instantly cut forward motor power and spin the robot away, completely overriding any commands sent by the Pi 5.

5.2.2 The Hardware Failure: The Death of the Pi 5

During the development and testing phases, the system encountered a catastrophic hardware failure. The computational load (running the 1-Billion parameter Llama model alongside the 10Hz navigation mathematics) had been highly optimized and carefully calculated to operate safely within the Pi 5's limits using a standard heatsink. However, the Pi 5 was completely fried due to a faulty buck converter in the power distribution pipeline. This electrical surge destroyed the board, leaving the robotic body (the Pi 4 and its attached hardware) without a cognitive or navigational host, and severing its primary spatial sensor (the LiDAR).

5.2.3 The Pivot: Laptop (RTX 5070 Ti) Migration

To save the project and restore Kai's functionality, the architecture was rapidly pivoted. The Tier 1 Master Brain and Tier 2 Navigation Engine were decoupled from the onboard Linux environment and migrated to a local Windows Laptop.

While initially a setback, this pivot provided a massive, unexpected performance upgrade:

- Tier 1 (Master Brain): Now leverages the laptop's RTX 5070 Ti GPU. This allows Kai to run the much larger and more robust Llama 3.1 model via Ollama with near-instant inference times. Crucially, the massive increase in compute overhead enabled the replacement of the linear OPCODE system with a full LangGraph multi-agent orchestration system (Supervisor, Analyst, Navigator, Caregiver)—a cognitive architecture that would have been impossible on the original Pi 5. Additionally, the ReSpeaker microphone array was physically relocated from the Pi 4 and plugged directly into the laptop via USB to centralize the audio pipeline on Tier 1.
- Tier 2 (Navigation Engine): Now executes the 10Hz kinematics and Bug Algorithm on the laptop's CPU, communicating with the Pi 4 over a local Wi-Fi network rather than the previous direct Ethernet link.
- Tier 3 (Edge Hardware Portal): The Pi 4 remains on the robot chassis. With the Pi 5 gone, the Pi 4 absorbed the USB LiDAR management duties. It acts as an ultra-low-latency physical proxy that streams 360-degree telemetry to the laptop and autonomously executes motor and safety commands.

5.3 Identified Software Integration Failures

Following the hardware pivot to the Laptop-Pi 4 hybrid setup, system integration testing revealed several critical software bottlenecks:

1. The ZMQ "Death Spiral" (Concurrency Lockup):
 - Issue: The communication bridge over Wi-Fi used ZeroMQ (REQ/REP). These sockets require a strict request-then-reply cycle. If the Pi 4 dropped a packet or momentarily disconnected due to Wi-Fi jitter, Tier 2 on the laptop would block indefinitely waiting for a reply. Consequently, Tier 1's async event loop would also freeze waiting for Tier 2, completely paralyzing the robot's brain.
2. Brittle LLM Parsing & State Coupling:
 - Issue: The Navigator Agent relied on a hardcoded Python `.split("MODE:")` string parsing method. When the LLM added conversational formatting or varied its capitalization, the parser crashed.
 - Issue: Vacuum state and movement state were tightly coupled. Requesting only a vacuum change resulted in an invalid "none" movement state, causing the entire command to be dropped.
3. Audio Processing and Environmental Interference:
 - Issue (Echo Feedback): The laptop's microphone picked up the robot's own Text-to-Speech (TTS) output, causing an infinite loop where the robot interpreted its own responses as new commands.
 - Issue (Loud Room Deafness): In loud presentation environments, the Voice Activity Detection (VAD) algorithms never detected a moment of "silence," creating an infinite sentence recording.
4. LLM "Rule Leaking" and Context Disconnect:
 - Issue: Negative constraints ("STRICTLY FORBIDDEN from saying 'Kai'") caused the Caregiver LLM to act erratically and vocalize its own system prompt restrictions. Furthermore, the Caregiver lacked awareness of actions taken by the Navigator, leading it to ask for clarification after a physical action had already been executed.

5.4 The New Architecture & Solutions Implemented

To resolve these post-pivot issues, the software architecture was refactored for resilience, thread safety, and modularity.

1. Self-Healing Communications
 - Async Timeouts: Replaced blocking ZMQ calls in Tier 1 with `asyncio.wait_for`. If the Pi 4 is offline, the Brain now gracefully handles the timeout and informs the user via TTS rather than freezing.
 - Socket Recreation: Implemented a self-healing pattern in Tier 2. If the Pi 4 times out over Wi-Fi, Tier 2 safely destroys and recreates its ZMQ REQ socket to clear corrupted state machines and seamlessly resume operation.

2. Robust Parsing & Decoupled State

- **RegEx Engine:** Upgraded Tier 1's Navigator Agent to extract commands using Regular Expressions (re.search), making the system immune to LLM formatting variations.
- **Decoupled Kinematics:** Separated the vacuum Boolean from the movement state machine. Added distinct granular directional modes (forward, backward, left, right) alongside wander, allowing the LLM to command complex sequences or change the vacuum state while leaving movement unchanged.

3. Advanced Audio & Multi-Modal Input

- **VAD Sweet Spot Tuning:** Adjusted the post_speech_silence_duration to 0.4s. This forces the STT engine to execute commands the millisecond the user takes a breath, bypassing the need for true silence in a loud room.
- **Anti-Echo Buffer:** Added a thread-safe 0.5-second sleep buffer to the TTS engine after speaking, preventing the microphone from ingesting trailing speaker audio.
- **Hot-Swappable Input Modes:** Integrated the keyboard module with global thread-safe hooks. Pressing Ctrl+T engages Text Mode, which invokes a strict set_mute_state(True) function to instantly kill background audio transcription and clear the terminal for silent, typed commands. Ctrl+V seamlessly restores voice functionality.

4. Context-Aware Cognitive Routing

- **Positive Reinforcement Prompts:** Removed strict negative constraints from the Caregiver to prevent conversational leaking.
- **Action Injection:** The Navigator now injects explicit success strings (e.g., "HARDWARE ACTION TAKEN: Successfully set...") into the Caregiver's context window. The Caregiver is explicitly instructed to trust this telemetry and simply relay the success to the user, eliminating conversational confusion.

5. Hierarchical Memory & Global Ledger System (Migrated & Scaled)

- A cornerstone of the system that carried over from the original Pi 5 design is the two-stage hierarchical memory architecture. Originally designed to protect the Pi 5's limited RAM, it proved equally vital on the laptop for preventing GPU VRAM exhaustion and managing unbounded context windows within the new LangGraph orchestration.
- **Short-Term Agent Logging:** During active execution, whenever the Supervisor routes tasks to the Analyst, Navigator, or Caregiver, each specific agent appends a brief summary of the action it performed into a rolling short-term context log. This dynamically expanding log ensures that every new agent added to the orchestration seamlessly contributes to Kai's immediate situational awareness.
- **The 30-Minute Global Ledger Compaction:** To maintain efficiency, an asynchronous temporal daemon monitors the timeline. Every 30 minutes, this system prompts the LLM to ingest the entire short-term log and compress it into a dense, high-level summary of the session's events.
- **JSON File Storage:** This compressed summary is then written as a single, chronological entry into an external "Global Ledger" saved as a local JSON file. Following the write, the short-term active RAM log is safely flushed. This elegantly solves the infinite context window problem, drastically freeing up resources while simultaneously augmenting the robot's long-term memory capabilities by providing a permanent, searchable history of interactions across multiple days and sessions.

5.5 Systems Synergy: How It All Works Together

The finalized hybrid system operates as a fluid, highly resilient pipeline:

1. **Ingestion (Laptop):** The user speaks (or types via Text Mode). The AudioToTextRecorder utilizes AI VAD to quickly isolate the sentence and passes it to the stt_wake_word_callback in Tier 1.

2. Cognitive Routing (Laptop/RTX 5070 Ti): The LangGraph Supervisor node intercepts the text. It analyzes keywords to determine if the intent is conversational, analytical (health/camera), or navigational.
3. Action Determination (Laptop): If navigational, the Navigator Agent (Llama 3.1) extracts the precise MODE and VAC state using RegEx. By acting as an autonomous reasoning engine, it can independently decide when to vacuum based on the interaction. It then appends its actions to the short-term memory log.
4. Network Transmission (Wi-Fi): The command is sent via ZMQ to Tier 2. Tier 2 translates this into differential wheel speeds and forwards it over the local Wi-Fi network to Tier 3 on the robot. Extensive testing confirmed that the Wi-Fi bridge introduced zero noticeable latency, fully preserving the real-time responsiveness of the system.
5. Safety Override (Pi 4): On Tier 3, if the onboard LiDAR detects an obstacle within 15cm, the Pi 4's hardware reflex instantly severs the ZMQ motor command and autonomously spins the robot away. Tier 2 on the laptop detects this via 50Hz telemetry and safely halts its own pathfinding until the reflex clears.
6. Synthesis & Storage (Laptop): The hardware confirmation flows back up to Tier 1. The Caregiver Agent ingests the context of the successful action, cross-references its short-term logs and the persistent JSON Global Ledger, and generates a natural, contextual, first-person response via pytt3x3. Concurrently, if critical health metrics trigger the emergency protocol, the WhatsApp integration fires independently. Every 30 minutes, these short-term logs are permanently archived into the Ledger, guaranteeing an unbroken chain of memory and safety.

6 WhatsApp, Wellbeing Assistant, and Calendar Integration

6.1 Purpose and Requirements

A WhatsApp-based interface was developed to provide a simple communication channel between the user, caregiver and robotic companion. WhatsApp was selected instead of a dedicated mobile application because it is familiar to most users, requires no additional installation and allows a caregiver to interact with the system remotely.

The subsystem was required to:

- receive commands through WhatsApp;
- provide live weather information and practical advice;
- create events in Apple Calendar;
- display upcoming reminders;
- send an automatic WhatsApp reminder before an event;
- hide reminders after they have been issued or after their event time has passed;
- restrict access to authorised WhatsApp numbers;
- provide a simple emergency stop command.

The final user commands were deliberately kept short:

```
help
weather
calendar
stop
add event [event name] at [time]
```

Examples include:

```
add event Doctor appointment at 2026-06-18 15:30
add event Walk at tomorrow 10:00
add event Take medicine at today 20:00
```

The semi-structured command format was selected because it was easier to parse reliably than fully unrestricted natural language while remaining understandable to a non-technical user.

6.2 System Architecture

The original intention was to run the service on the Raspberry Pi 5. After the Pi 5 hardware failure, the software was migrated to a teammate's Windows laptop. Development and testing were completed on macOS before the project folder was transferred to Windows.

The final communication path was:

```
WhatsApp user
|
Meta WhatsApp Cloud API
|
ngrok HTTPS tunnel
|
FastAPI server on Windows laptop
|
Weather, Apple Calendar and reminder modules
|
WhatsApp response
```

Meta requires a public HTTPS webhook, whereas the FastAPI server runs locally on port 8000. ngrok was therefore used to expose the local server through a secure public URL.

The main software files were:

- `robot_api.py`: FastAPI routes, webhook processing, command handling and automatic reminders;
- `weather_tools.py`: live weather retrieval and advice generation;
- `apple_calendar_tools.py`: Apple iCloud CalDAV connection and event creation;
- `pending_calendar_reminders.json`: persistent local reminder records;
- `.env`: access tokens, phone numbers, calendar credentials and location settings;
- `setup_windows.bat` and `run_windows.bat`: Windows installation and startup scripts.

Sensitive credentials were stored in the `.env` file rather than embedded directly in the source code. These included the Meta access token, WhatsApp phone-number ID, authorised user numbers, Apple account email and Apple app-specific password.

6.3 Command Processing, Weather, Calendar and Reminder Functions

6.3.1 Webhook and Command Handling

Incoming messages are delivered by Meta to the FastAPI `/webhook` endpoint. The webhook extracts the sender number, message type and text body from the incoming JSON data. Delivery and read-status notifications do not contain a user message, so these are detected and ignored.

For a valid text message, the command handler checks the sender against the `AUTHORIZED_USERS` configuration. The command is then routed to the appropriate function. The generated response is sent back through the WhatsApp Cloud API.

The server prints the incoming message, generated reply, HTTP status code and Meta response. This logging proved important during integration because a webhook could return HTTP 200 even when the outgoing WhatsApp reply had failed.

The `stop` command was retained as a basic safety command. In the complete robot architecture it is intended to connect to the movement subsystem and immediately cancel motion.

6.3.2 Weather Advice

The `weather` command retrieves live information from the Open-Meteo API. The service returns current temperature, precipitation, weather condition, wind speed and daily rain probability.

Instead of returning only raw values, the system converts the forecast into practical advice. For example:

- if rain is detected or the rain probability is high, the user is advised to take an umbrella;
- if the temperature is low, the user is advised to wear a jacket or coat;
- if the temperature is high, the user is reminded to drink water;
- if wind speed is high, the user is warned to take care outdoors.

This feature was included because the project was intended as an eldercare companion. The goal was therefore not only to report data, but to translate the data into a useful daily recommendation.

6.3.3 Apple Calendar Event Creation

Apple Calendar events are created using the iCloud CalDAV service. Authentication requires the Apple account email address and an app-specific password. The normal Apple account password is not accepted by the CalDAV interface.

The event parser supports three time formats:

- today HH:MM;
- tomorrow HH:MM;
- YYYY-MM-DD HH:MM.

For the command

```
add event Take medicine at today 20:00
```

the software extracts the event title and constructs a timezone-aware London datetime. The calendar module then creates a one-hour event in the selected iCloud calendar.

The Apple Calendar connection was tested independently before the WhatsApp interface was used. This separated calendar authentication failures from webhook and parsing errors. The test command was:

```
python -c "from apple_calendar_tools import add_calendar_event;
print(add_calendar_event('Test calendar event',
'2026-06-18T15:30:00+01:00', 30))"
```

This direct test revealed that one major failure was an iCloud authorisation error. The issue was resolved by generating and using a valid Apple app-specific password and confirming the exact calendar name returned by iCloud.

6.3.4 Reminder System

After an event is successfully created in Apple Calendar, a second lightweight record is written to `pending_calendar_reminders`. A typical record is:

```
{
  "title": "Take medicine",
  "start_time": "2026-06-18T20:00:00+01:00",
  "sent": false
}
```

This local record does not replace Apple Calendar. It provides a reliable queue for automatic reminders without requiring the system to repeatedly query iCloud.

An asynchronous reminder loop starts with the FastAPI application. Every 10 seconds it checks the stored reminders. If an unsent event falls within the next 10 minutes, the system sends a WhatsApp message such as:

```
Reminder: Take medicine is coming up at 20:00.
```

The reminder is then marked as sent. The calendar command displays only pending future reminders. Events are excluded when:

- the reminder has already been sent; or
- the event start time has already passed.

This prevents completed or obsolete reminders from continuing to appear in the user's upcoming list.

A 10-minute reminder window was selected for demonstration purposes because it allowed the automatic notification to be shown without a long wait. In a production version this value should be configurable.

6.4 Problems Encountered and Design Changes

Several problems were identified during development.

6.4.1 Raspberry Pi 5 failure

The initial deployment used FastAPI inside a Docker container on the Raspberry Pi 5. When the Pi 5 failed, the service was moved to a Windows laptop. The user-facing functions did not change; only the hosting platform changed.

6.4.2 Docker and ngrok networking

When ngrok ran on the Pi host but FastAPI ran inside Docker, forwarding to `localhost:8000` produced HTTP 502 errors. The host and the container did not share the same localhost interface. During diagnosis, ngrok had to be pointed to the Docker container's IP address.

Removing Docker from the final Windows demonstration simplified this architecture. FastAPI and ngrok both ran directly on the laptop.

6.4.3 Misleading event error message

Initially, command parsing and Apple Calendar creation were contained in the same exception handler. As a result, a valid command could produce an incorrect “please use this format” message when the actual failure was calendar authentication.

Detailed exception logging was added so that parsing errors, calendar errors and networking errors could be distinguished.

6.5 Testing Procedure

Testing was performed progressively.

First, the source files were checked for syntax errors:

```
python -m py_compile robot_api.py
python -m py_compile weather_tools.py
python -m py_compile apple_calendar_tools.py
```

The local FastAPI server was then started using:

```
python -m uvicorn robot_api:app --host 0.0.0.0 --port 8000
```

Local endpoints were tested with:

```
curl http://localhost:8000
curl http://localhost:8000/weather
curl http://localhost:8000/calendar/upcoming
```

The public ngrok endpoint was tested before connecting it to Meta. The webhook verification route was also checked using the configured verification token.

The final end-to-end demonstration sequence was:

1. send help;
2. send weather;
3. send an event command with a time within the next 10 minutes;
4. verify that the event appears in Apple Calendar;
5. send calendar and confirm that the reminder appears;
6. wait for the automatic WhatsApp reminder;
7. send calendar again and confirm that the event has disappeared;
8. send stop.

6.6 Handover and Reproduction

To reproduce the subsystem on the Windows demonstration laptop:

1. copy the complete project folder from the development Mac;
2. install Python 3.11 or Python 3.12;
3. run `setup_windows.bat`;
4. create the `.env` file from the supplied template;
5. enter the current Meta access token and Apple app-specific password;
6. run `run_windows.bat`;
7. start ngrok on port 8000;
8. enter the ngrok webhook URL in the Meta developer configuration;
9. subscribe the webhook to the `messages` field;
10. test the weather, calendar, reminder and stop commands.

Before the demonstration, the operator should confirm that the Meta token has not expired, the selected iCloud calendar name is correct, the authorised telephone number contains no leading plus sign and the laptop will not enter sleep mode.

6.7 Limitations and Future Work

The final system met the demonstration requirements but retained several limitations:

- the Meta temporary access token may expire;
- ngrok must remain active throughout operation;
- the reminder interval is fixed at 10 minutes;
- event deletion and editing are not yet supported through WhatsApp;
- the JSON reminder file is intended for a single user;
- the calendar service depends on internet access and valid iCloud credentials;
- the stop command must remain connected to the physical motor-control safety layer.

Future improvements could include configurable reminder times, recurring medication events, caregiver escalation, missed-reminder acknowledgement, database-backed storage and deployment to a permanent hosted server. “

7 Health & Activity Monitoring

7.1 Camera Vision

7.1.1 The Sony IMX500 AI Camera

The vision input for the edge node is provided by the Raspberry Pi AI Camera, which is built around Sony's IMX500 intelligent vision sensor. Unlike a conventional camera module, the IMX500 integrates a dedicated image signal processor (ISP) and a neural network accelerator on a single die, with on-package SRAM for storing model weights. Once the sensor begins streaming, the ISP performs basic image processing on each raw frame and feeds the processed frame directly into the AI accelerator. Image processing therefore occurs entirely on-sensor, and the IMX500 sends both an image stream and a structured per-frame tensor (bounding boxes, class indices, confidence scores) to the host Pi over the standard interface.

The practical consequence is that the host Pi 4 never sees uncompressed pixel data passing through the neural network. The only data crossing the CSI bus for the inference path is a small metadata structure of detections per frame. Model weights are uploaded once at boot as an `.rpk` file. This first-time upload is non-trivial (up to approximately two minutes for the YOLO11n model used), but the cost is paid once per camera-acquisition lifetime; subsequent inference runs at the configured frame rate with no further upload overhead.

7.1.2 Design Rationale

A conventional approach to vision on a Pi 4 would stream raw frames from a camera module to the host CPU and run inference there using a lightweight model on TensorFlow Lite or ONNX Runtime. Instead, we went with the IMX500 for three reasons:

1. **CPU budget:** The Pi 4 in this node is already responsible for enough other heavy tasks like the LiDAR safety-reflex loop, BLE heart-rate streaming, and the ZMQ communications layer to the upstream tiers. Running a YOLO-class model on the same cores would compete directly with timing-sensitive safety code. Off-loading inference to the sensor itself keeps the host CPU available for control-loop work.
2. **Inference latency:** Because the neural network is located on the sensor's front-end, there is no PCIe, USB, or CSI transfer between pixel capture and detection output. Detections arrive as metadata on the same frame they were computed from, eliminating the latency that dominates pipelines on a Pi.
3. **Bandwidth and privacy:** The way the IMX500 is used only allows it to stream metadata to the Pi 4. Raw pixels need not leave the sensor package, which is relevant both for CSI bus utilisation and for downstream privacy claims about what data the system retains.

The trade-off is reduced model flexibility: only models that have been converted to the IMX500's quantised `.rpkl` format and that fit within the on-package SRAM can run on-sensor. This is acceptable for our use case, where person detection is sufficient.

7.1.3 Software Architecture on the Pi 4

The camera is exposed to the rest of the edge node through a single class, `CameraDetector`, which wraps `PiCamera2`. The class is responsible for four things: model and label loading, camera configuration, firmware-load synchronisation, and output parsing.

Lifecycle: Each `CameraDetector` instance follows a strict acquire-use-release pattern. Construction performs the IMX500 firmware upload synchronously; this is wrapped in a 180-second watchdog implemented with a worker thread plus `threading.Event`, so a stalled firmware load surfaces as a `RuntimeError` rather than an indefinite hang. The release path `CameraDetector.close()` calls both `picam2.stop()` and `picam2.close()` on the underlying `PiCamera2` instance. The distinction is important: `stop()` alone halts streaming but leaves the camera in `libcamera's Configured` state, which causes a subsequent `acquire()` call from any later instance to fail. `close()` returns the camera to the `Available` state, allowing the on-demand pattern described below to work across multiple snapshots in a single process lifetime.

Inference output handling: The IMX500 only attaches inference results to some frames, not all of them. This means a single call to `capture_metadata()` could return “no AI output yet,” an empty detection list, or a stalled camera, all of which look the same from the outside. `CameraDetector._latest_detections()` keeps reading frames until one carries inference output, returns its detections (an empty list is a valid result), and gives up after 5 seconds if no frame ever arrives.

Detection parsing: Two parser functions are provided for the two output formats produced by the supported model families. `parse_yolo` handles YOLO-style outputs (centre-x, centre-y, width, height in normalised 640×640 coordinates), and `parse_efficientdet` handles EfficientDet's 1-indexed class IDs and $(y_{\min}, x_{\min}, y_{\max}, x_{\max})$ box format in 320×320 coordinates. Both rescale to the configured main stream resolution (default 1920×1080) and emit a uniform output schema. Both filter in-line to retain only the `person` class, since downstream consumers in this node have no use for other COCO classes; the class index is resolved at construction time from the labels file, with a built-in COCO-80 list as fallback.

Stance heuristic: Detections are annotated with a stance estimate (`standing`, `sitting`, `lying_down`) derived from the bounding-box aspect ratio. This is explicitly not a substitute for proper pose estimation, but provides a cheap first-pass signal usable as a trigger for follow-up behaviour (e.g. a sustained `lying_down` detection feeding into a fall-detection upstream).

7.1.4 Integration with the Edge Node

The edge node uses the camera in an on-demand pattern: a new `CameraDetector` is constructed when a snapshot is requested, used to produce exactly one detection result, and released via `close()` in a `finally` block before the function returns. The alternative of keeping a single persistent camera instance warm across the process's lifetime was implemented and benchmarked, but rejected for this deployment: the

cost of holding the IMX500's full frame buffers and loaded network in memory during long idle periods outweighs the latency win of skipping firmware reload, given that snapshot requests are infrequent compared with the constant-throughput sensors on the same Pi.

Snapshot requests reach `capture_single_snapshot()` through a ZMQ REP handler bound to the upstream tier's command channel. When the laptop sends a camera command, the handler acquires a `threading.Lock` to prevent multiple calls on an already acquired camera, runs the full acquire-snapshot-release cycle described above, and replies with a JSON payload. The payload contains the detection list, a `person_detected` boolean, and a status code. Because the handler is synchronous (it blocks the ZMQ loop until the camera releases) the laptop is naturally rate-limited to one in-flight request at a time without any additional flow-control logic, and the structured response shape makes integration with the upstream tier straightforward.

7.2 Heart Rate Monitor

7.2.1 Decathlon HRM Chest Strap

Heart rate input is provided by a Decathlon-branded Bluetooth Low Energy arm band. The specific strap deployed here advertises with the local name *HRM BAND 41244-17* at MAC address `E2:BF:84:6C:13:68`, both of which are used by the Pi-side scanner to disambiguate it from other heart-rate-capable devices that may be in radio range.

7.2.2 BLE Protocol

The strap exposes the Heart Rate Service as a GATT primary service with a single notify-capable characteristic, `0x2A37`. The host subscribes to notifications on this characteristic at connection time; the strap then pushes a small binary frame (typically two or three bytes) every time it produces a new measurement, at roughly 1 Hz under normal operation.

The parse function on the Pi inspects the flag byte and parses the data to send back the current heart rate in BPM.

7.2.3 Software Architecture on the Pi 4

The BLE stack on the Pi 4 is built on `bleak`, an asyncio-native cross-platform BLE library, running BlueZ underneath on Linux. The full heart-rate pipeline is implemented as a single asyncio routine, `ble_heart_rate_loop()`, which executes the following state machine indefinitely:

Scan and filter: A scan is initiated via `BleakScanner.discover()` with a service-UUID filter, restricting results to devices advertising the Heart Rate Service. The result list is then filtered through `is_target_band()`, which matches on MAC address first (exact, case-insensitive) and falls back to a substring match on the advertised device name. The address-first matching ensures correctness even in environments where multiple heart-rate-capable devices are present.

Connect and subscribe: Once the target device is identified, a `BleakClient` is constructed and entered as an async context manager. After a short stabilisation delay, the client subscribes to notifications on the Heart Rate Measurement characteristic with `start_notify()`, passing a callback that parses incoming frames and updates a shared `GLOBAL_STATE` dictionary with the latest BPM value.

Disconnection handling: A `disconnected_callback` is registered on the `BleakClient` at construction. When the strap disconnects, the callback sets an `asyncio.Event` that the routine is awaiting. The connection block then exits cleanly via the context manager, the global state is reset to "unknown", and the loop re-enters the scan phase.

Bluetooth-stack recovery: A failure-counting policy sits above the connection loop. Consecutive scan or connection failures are tracked, and after a configurable threshold (`BT_RESET_AFTER_FAILURES = 3` in the current deployment), the loop issues `sudo systemctl restart bluetooth` via `asyncio.to_thread` before retrying. This handles the case where BlueZ enters a state in which scans return empty result sets indefinitely until restarted. The failure counter is reset on any successful connection.

7.2.4 Integration with the Edge Node

The latest heart rate value is exposed to the upstream tier through the same ZMQ REP channel that handles the camera command. When the laptop sends a `get_heart_rate` command, the handler replies with a JSON payload containing both the current BPM (or the string "unknown" if the band is not currently connected) and a `connected` boolean. This explicit two-field response is preferred over a single value alone: it allows the upstream recipient to distinguish “the band is connected and the user has a heart rate of zero” from “the band is not currently connected and we therefore have no data”.

7.2.5 Future Extensions

- **Heart rate variability (HRV) and RR intervals:** The Heart Rate Measurement characteristic carries RR-interval data (time between successive beats). If the Decathlon band reports these, the parser could be extended to extract them and compute RMSSD or pNN50 over a rolling window. HRV trends are a substantially richer signal for stress and recovery state than instantaneous BPM, and would feed naturally into the wellbeing-monitoring tier.
- **Push-based delivery for derivative signals.** While instantaneous BPM is appropriately pull-driven, threshold-crossing events (sustained tachycardia, sudden bradycardia, loss of skin contact) are inherently asynchronous and would be better served by a ZMQ PUSH socket from the Pi to the laptop. The existing sensory-interrupt channel used for voice commands is the obvious carrier.
- **Cross-modal fusion.** Heart rate, when correlated with the camera’s stance estimate and the LiDAR-derived motion state, would enable richer wellbeing inferences than any single signal in isolation, distinguishing for example, “resting on the sofa” (low BPM, lying down, stationary) from “unresponsive after a fall” (low BPM, lying down, stationary, but with a sudden transition rather than a gradual one). This is best implemented at the upstream wellbeing tier, since it requires consolidating signals from multiple edge subsystems that the Pi 4 deliberately does not couple internally.

8 Movement Subsystem & Navigation Integration

8.1 Design Evolution, URDF, & Simulation Phase

The initial conceptualization of the project involved a quadrupedal “robot dog” design. However, as payload requirements were finalized—specifically the inclusion of a high-draw 12V DC vacuum cleaner motor and heavy-capacity batteries—the system architecture was revised to a differential wheeled chassis. Wheeled architectures provide superior load-bearing capabilities compared to articulated legs, which require continuous torque to maintain a standing posture.

Prior to physical hardware integration, the robot’s kinematic models and navigation stack required testing in a simulated environment. This necessitated the authoring of a custom URDF (Unified Robot Description Format). Establishing the correct TF (Transform) tree was a critical requirement; precise spatial relationships and coordinate frames had to be mapped between the `base_link`, the individual `wheel_links`, and the `laser_frame` to ensure sensor data accurately reflected the physical configuration.

Furthermore, calculating accurate moments of inertia for a chassis with an off-center vacuum payload was necessary to prevent the simulated robot from exhibiting erratic physics in Gazebo. Once the URDF accurately mirrored the physical constraints, the model was deployed into Gazebo for physics simulation and RViz2 for spatial visualization. This initial configuration phase allowed for the pre-testing of costmap generation, sensor fusion, and Nav2 path-planning behaviors in an idealized environment before physical deployment.

8.2 Hardware Sourcing & Specifications

The physical movement subsystem was constructed using high-torque components to accommodate the loaded chassis. To achieve stable differential steering, a tricycle-style wheel configuration was selected.

Component	Specification / Details	Purpose & Rationale
Drive Motors	2x Pololu 37D Metal Gearmotors w/Encoder - 12V 83RPM 45Kg.cm	Provides the required torque for hauling the chassis, battery, and vacuum payload.
Motor Mounts	Pololu Stamped Aluminum L-Bracket Pair	Engineered specifically for 37D motors to ensure rigid mounting under load.
Drive Wheels	Pololu Wheel 90x10mm Pair (Yellow)	Mated to the D-shafts of the front drive motors to provide required ground clearance and forward traction.
Rear Castor	Tente Swivel Castor (100kg Cap, 50mm, Thermoplastic Rubber)	Mounted at the rear to support the bulk of the chassis weight while allowing low-friction pivot capability for differential steering.
Motor Driver	Cytron MDD10A (2-Channel)	High-current H-bridge driver capable of handling continuous current draw without thermal throttling. Operated via standard PWM and DIR logic signals.
Payload Control	Logic-Level MOSFET Switch Module	Utilized to actuate the 12V vacuum motor. Features an onboard flyback diode to safely dissipate inductive kickback.
Vision/Sensors	RPLidar A1	A 360-degree 2D laser scanner interfacing via a dedicated USB serial adapter, sourced for mapping and localization.

Table 4: Movement and Payload Hardware Specifications

8.3 Initial Software Architecture

The primary logic controller for the platform was a Raspberry Pi 5. To eliminate cross-dependency issues and ensure a reproducible environment, the software stack was built on Ubuntu 24.04 running entirely inside a Docker container. The intended navigation architecture relied on the Nav2 framework paired with the SLAM toolbox, acting on high-level commands dispatched via a LangGraph LLM integration. The RPLidar A1 and wheel encoders were configured to feed environmental and odometry data into the Pi 5 to facilitate autonomous path planning.

8.4 Hardware Failure & Contingency Implementation

During final integration, the primary Raspberry Pi 5 experienced an unexpected hardware failure and ceased functioning. The project was migrated to a backup Raspberry Pi 4, which was limited to 1GB of RAM. It was determined that this memory capacity was insufficient to reliably host the Docker container alongside the computationally expensive SLAM and Nav2 processes. Consequently, an immediate system pivot was required to bypass the Nav2 stack entirely.

To maintain functional movement, positional awareness, and LLM integration, a custom ROS2 navigation architecture was developed. This fallback system consisted of three primary files: an `odom` node for kinematic calculations (`odom.py`), a main controller (`robot_brain.py`) for decision-making, and a start script to initialize the subsystem.

8.4.1 Odometry & Encoder Kinematics

The `odom` node calculated the robot's position and heading utilizing the ticks from the wheel encoders. The magnetic encoders generated discrete pulses corresponding to the rotation of the motor shaft. By measuring the specific number of counts per revolution (CPR) required for a full 360-degree (2π radians) rotation of the wheel, the raw ticks were converted into linear displacement.

The center distance traveled (Δd) and change in heading ($\Delta\theta$) over a given wheelbase (L) were continuously integrated to update the global position (x, y) and orientation (θ) relative to the starting point:

$$x_{new} = x_{old} + \Delta d \cdot \cos\left(\theta_{old} + \frac{\Delta\theta}{2}\right) \quad (1)$$

$$y_{new} = y_{old} + \Delta d \cdot \sin\left(\theta_{old} + \frac{\Delta\theta}{2}\right) \quad (2)$$

$$\theta_{new} = \theta_{old} + \Delta\theta \quad (3)$$

8.4.2 Main Controller & Bug Algorithm Bypass

The `robot_brain.py` node functioned as a state machine, replacing Nav2 by subscribing to the odometry, LiDAR, and camera topics, and publishing velocity vectors. It utilized five distinct operational states:

- **State 0 (Vacuum Wander):** Default exploration state, commanding forward velocity while actively powering the vacuum payload.
- **State 1 (Non-Vacuum Wander):** Exploration state where the chassis maintains forward movement, but the vacuum payload is deactivated to conserve power and reduce acoustic interference.
- **State 2 (Stop):** Immediate halt triggered by camera alerts or LangGraph interrupts. Disables the vacuum to allow for clear voice interactions.
- **State 3 (Return Home):** Utilizes the dead-reckoned odometry to calculate a return vector to the origin (0,0). The heading error is calculated using continuous angle error normalization across the $-\pi/\pi$ boundary layer:

$$e_\theta = \text{atan2}(\sin(\theta_{target} - \theta_{current}), \cos(\theta_{target} - \theta_{current})) \quad (4)$$

- **State 4 (Bug Algorithm Bypass):** A localized reactive obstacle avoidance sequence. If the LiDAR detects an obstruction directly ahead (under 0.5 meters) while homing, the robot calculates the clearest turning direction, executes a timed 90-degree lateral bypass maneuver to clear the obstacle, and subsequently recalculates its vector to the origin.

This manual, reactive architecture successfully accommodated the memory constraints of the Pi 4, ensuring the movement subsystem remained operational and spatially aware for the final deliverable.

9 Design Decisions and Rationale

Throughout the development of KAI 9, several important engineering decisions were made to balance functionality, reliability, manufacturability and cost. Table 5 summarises the key decisions and the reasoning behind them.

Table 5: Major Design Decisions and Rationale

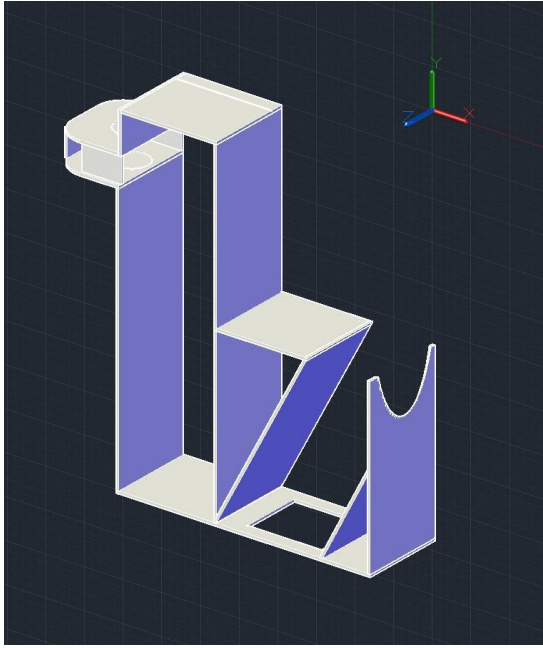
Design Decision	Rationale
6 mm Birch Plywood Chassis	Provided structural rigidity whilst supporting the battery, vacuum system and electronics. It was also inexpensive, easy to manufacture and allowed components to be securely mounted using screws.
Component-Centred Internal Layout	The chassis was designed around the battery and vacuum assembly to maximise space utilisation, improve weight distribution and simplify assembly.
Two-Wheel Differential Drive	Two powered front wheels and a rear castor were selected over tracks to reduce torque requirements while maintaining manoeuvrability and supporting the robot's payload.
3D Printed Outer Shell	Enabled rapid prototyping, low-cost manufacture and the creation of complex geometries, including integrated wheel protection.
Dog-Inspired Appearance	A robotic dog form factor was chosen to encourage user engagement and create a friendly appearance. Functional elements were incorporated naturally into the design, such as the vacuum outlet forming the tail.
Sensor Placement Strategy	The camera was positioned within the eyes for a natural forward-facing view, the microphone within the mouth to reduce interference from the speaker and vacuum system, and the LiDAR at the highest point of the robot to prevent scan obstruction.
LiDAR-Based Navigation	LiDAR was selected as the primary navigation sensor due to its reliable obstacle detection and environmental awareness in varying lighting conditions.
Raspberry Pi 5 Compute Platform	Provided sufficient processing capability for AI workloads, navigation, sensor processing and API integration while remaining compact and easy to develop on.
Rear Charging Dock Interface	Dedicated mounting locations were incorporated at the rear of the robot to accommodate future charging plates. These were designed to align with charging pins on a docking station, enabling future autonomous self-charging capability.
Grey Colour Scheme	Grey filament was chosen due to its low cost, availability and ability to hide minor manufacturing imperfections whilst maintaining a professional appearance.

The decisions outlined above were made throughout the project lifecycle and were continually reviewed as subsystem integration progressed. The primary objective was to ensure that KAI 9 remained reliable, maintainable and capable of meeting the project's Minimum Viable Product requirements within the available budget and development timeframe.

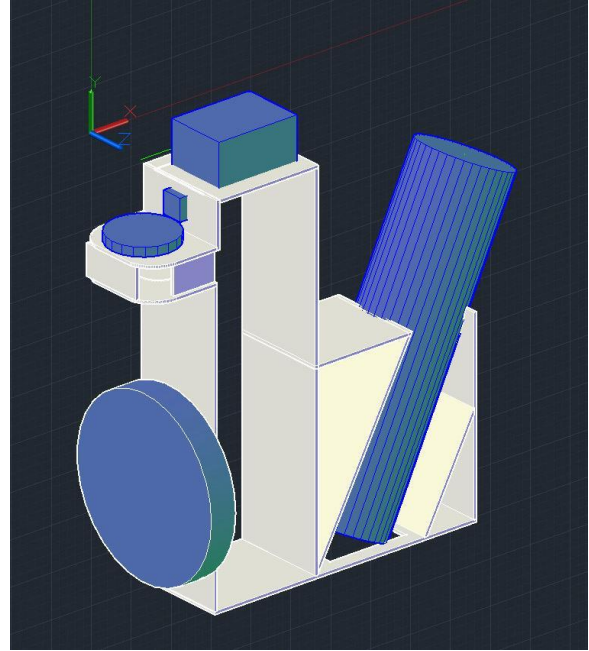
There were further ideas that were cut due to time constraints. One being ear-mounted LEDs. Future LED indicators would improve visibility of KAI-9 during low-light or night-time operation. Another idea was a bin mounted to the charging dock to allow an easy discard of waste for the user.

9.1 Chassis and Enclosure Design Iterations

The chassis design was developed with careful consideration of internal component placement. It was important to keep key items such as the vacuum microphone, speaker, and other embedded electronics in mind during the design process. These components needed to be incorporated as seamlessly as possible, ensuring they remain as inconspicuous as possible while still maintaining accessibility and functionality.



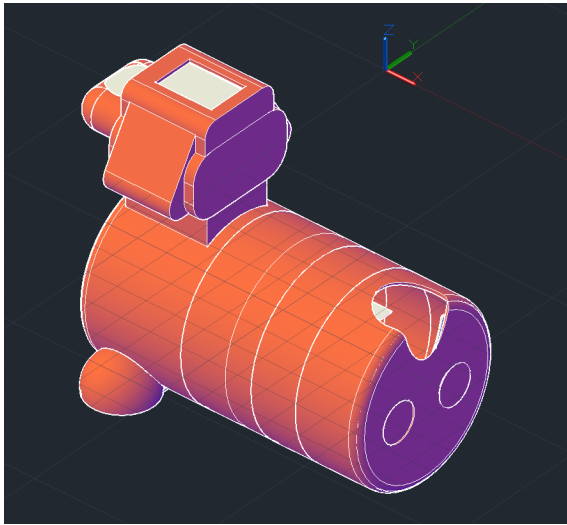
(a) Chassis design without internal components



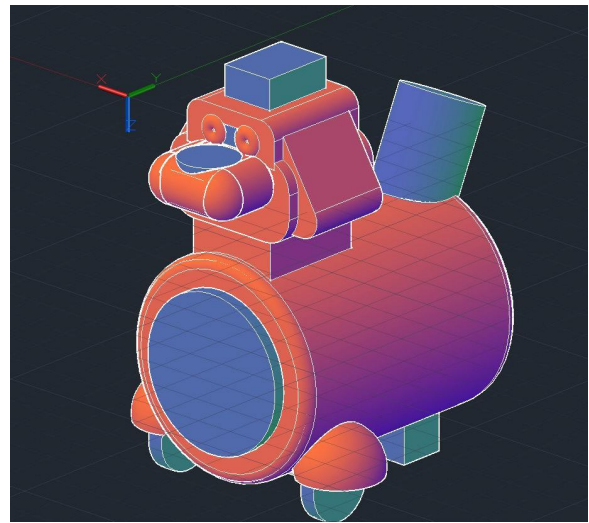
(b) Chassis with internal components (highlighted in blue)

Figure 4: Chassis design iteration showing empty structure versus integrated component layout

The next stage of development focused on refining the external shell to ensure a clean aesthetic while still accommodating all internal subsystems. The goal was to maintain a compact and visually minimal design while fully enclosing the hardware.



(a) 3D shell design without internal components



(b) 3D shell with internal components integrated

Figure 5: Final 3D enclosure design showing component integration and concealment strategy

9.2 Testing and Validation

The testing and validation phase was conducted incrementally throughout the development of KAI-9. As several hardware components had long lead times, software development and testing were prioritised during the early stages of the project. Individual software modules, including navigation, voice interaction, sensor integration and AI services, were developed and validated independently before the complete hardware platform became available.

As hardware components arrived, subsystem testing was expanded to include integration testing between the electronics, sensors and mechanical systems. This iterative approach allowed issues to be iden-

tified and resolved as the robot progressed from a digital prototype to a fully assembled physical platform.

Several design refinements were made following physical testing. During initial movement trials, it was observed that the rear castor wheel could become caught on the lower edge of the 3D printed chassis. To resolve this issue, the chassis was trimmed to provide additional clearance, resulting in smoother movement and improved manoeuvrability.

Accessibility improvements were also implemented. The side section of the neck assembly was opened to provide easier access to the main power switch, simplifying operation and maintenance. In addition, black painted accents were added to selected exterior features to improve the overall appearance of the robot and enhance visual contrast within the dog-inspired design.

Figure 6 illustrates the final modifications made following testing and validation activities.

The iterative testing process highlighted several opportunities for improvement that were not apparent during the design stage. By evaluating the robot under realistic operating conditions, both functional and aesthetic modifications were identified and implemented. These changes improved accessibility, reliability and the overall user experience while maintaining the intended appearance and functionality of KAI-9.

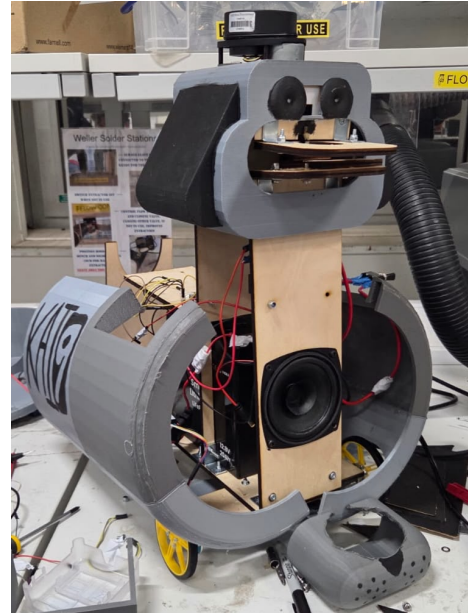


Figure 6: Assembly of KAI 9 for testing and validation.

9.3 Final Prototype Implementation



(a) Front view



(b) Side view

Figure 7: Final assembled system front view and side view

10 Materials and Bill of Materials

10.1 Overview

The KAI 9 prototype was developed using a combination of commercial off-the-shelf (COTS) components, custom fabricated mechanical assemblies and 3D-printed structures. Component selection was guided by the project's functional requirements, available budget, reliability requirements and compatibility with the Raspberry Pi 5 compute platform.

The hardware architecture consists of five primary subsystems:

- Compute and Processing
- Navigation and Sensing
- Mobility and Actuation
- Power Distribution and Charging
- Audio and User Interaction

Where possible, components were selected from reputable suppliers to minimise integration risks and ensure compatibility with the project's timeline.

10.2 Major Components

Subsystem	Component	Cost (£)
Compute	Raspberry Pi 5 (16GB)	225.93
Navigation	RPLiDAR C1	Department Supplied
Vision	Raspberry Pi Camera Module	65.00
Power System	20Ah LiFePO ₄ Battery	65.00
Audio Input	ReSpeaker USB Microphone Array	50.55

Table 6: Major System Components

10.3 Fabrication Materials

The robot chassis and supporting structures were manufactured using a combination of laser-cut birch plywood and additively manufactured components. Structural elements were fabricated using 6 mm and 3 mm birch plywood, while custom brackets and mounting fixtures were produced using 3D printing processes.

Additional materials included:

- M3 fasteners, nuts and washers.
- Motor mounting brackets and hubs.
- Electrical wiring (12 AWG and 16 AWG).
- Fuse holders and blade fuse distribution blocks.
- Power connectors and ring terminals.
- Pogo pins for charging dock contacts.

10.4 Power Distribution Components

A dedicated power distribution system was developed to safely manage battery power and subsystem protection. The power architecture incorporated:

- 20 Ah LiFePO₄ battery.

- Battery Management and Charging System.
- 30 A blade fuses.
- Fuse holders and distribution blocks.
- Buck converters for voltage regulation.
- Schottky diode protection circuitry.
- Emergency isolation switches.

These components were selected to ensure safe operation of the robot whilst supporting future subsystem expansion.

10.5 Total System Cost

The total value of procured hardware exceeded the original project budget allocation. This increase was primarily driven by the selection of higher-performance compute hardware, sensing systems and battery technologies required to achieve the project's autonomous navigation and companion functionality objectives.

The final prototype represents an estimated system value of approximately £900–£1,000 when all purchased components, fabrication materials and supplied equipment are considered.

10.6 Bill of Materials Traceability

A complete Bill of Materials (BOM), including supplier information, purchase links, quantities and procurement records, is maintained within the project repository and included in Appendix A of this Design History File. This ensures that future developers can source replacement components or reproduce the prototype with minimal additional research.

11 Future Work

Although the project successfully delivered the core Minimum Viable Product (MVP), several opportunities for further development were identified throughout the design and testing process. Due to the limited project duration, development efforts were prioritised towards achieving a robust and reliable implementation of the primary requirements. The following features are recommended for future iterations of the KAI 9 platform.

Table 7: Future Development Roadmap

Feature	Description
Advanced Conversation	More natural multi-turn conversations and contextual responses.
Routine Learning	Learn user habits and provide personalised assistance.
Voice Calling	Enable voice communication with family members and carers.
Health Monitoring	Medication reminders, activity tracking and wellbeing reporting.
Calendar Integration	Integration with Google and Apple Calendar services.
Mobile Application	Dedicated application for monitoring, configuration and notifications.
Personalised Voice	Support custom voice profiles, including family member voices.
Smart Home Integration	Connectivity with lighting, heating and home automation systems.
Mechanical Improvements	Improved docking reliability, weight reduction and chassis optimisation.

11.1 Conclusion

The KAI 9 platform provides a strong foundation for future development as an autonomous companion and sanitation robot. The modular architecture adopted during this project allows additional features to be integrated with minimal redesign of the existing system. Future work should focus on expanding the robot's conversational capabilities, health-monitoring functionality and ecosystem integration whilst maintaining the reliability and usability demonstrated by the current prototype.